



Digital Design Verification

Lab Manual # 23 – Control Hazards

Release: 1.0

Date: 25-June-2024

NUST Chip Design Centre (NCDC), Islamabad, Pakistan



Copyrights ©, NUST Chip Design Centre (NCDC). All Rights Reserved. This document is prepared by NCDC and is for intended recipients only. It is not allowed to copy, modify, distribute or share, in part or full, without the consent of NCDC officials.

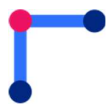
Revision History

Revision Number	Revision Date	Revision By	Nature of Revision	Approved By
1.0	25/06/2024	Ali Aqdas	Complete manual	-
2.0	18/08/2025	Umer Farooq / Fawad Ahmed	Instruction Updated	



Contents

Contents	2
Objective	3
Tools	3
Instructions	3
RISC-V Data and Control Paths	4
Pipelining: Control Hazards	4
Task	5
References	5



Objective

The objective of this lab is to

- Append the design with additional logic to handle control hazards.

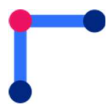
Tools

- Cadence Xcelium

Instructions

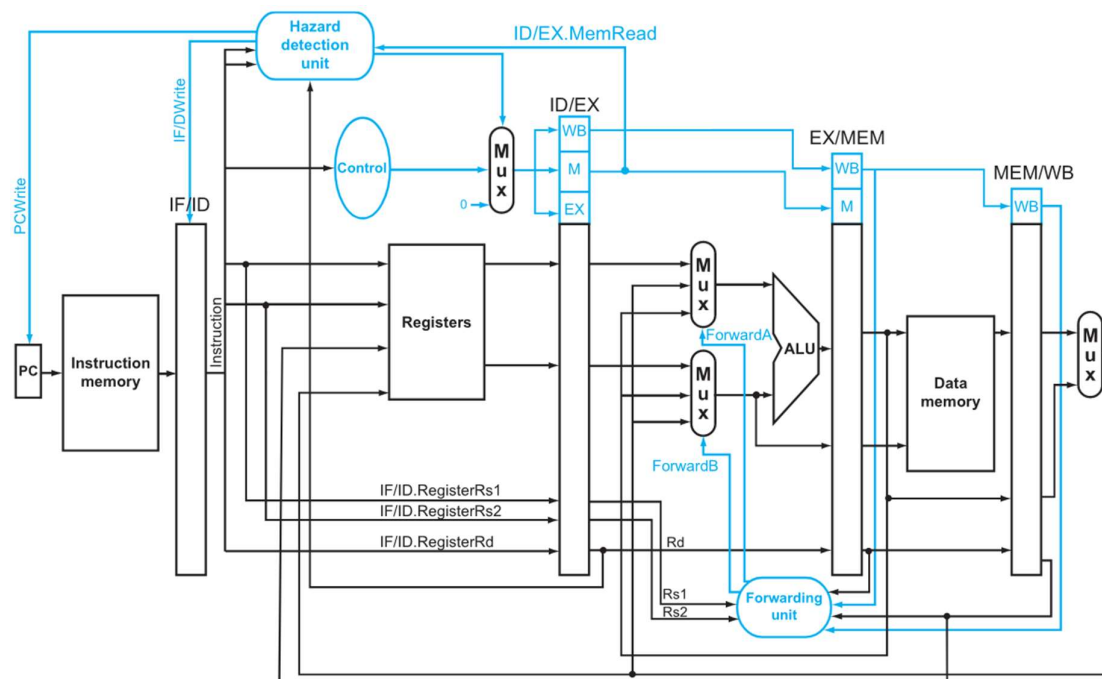
The students must design and submit the updated files for evaluation of this lab. **Additional modules may be needed for forwarding and hazard detection.**

./<student-name>/	./rtl/	program_counter.sv inst_mem.sv data_mem.sv reg_file.sv imm_gen.sv alu_logic.sv branch_comp.sv control_unit.sv IF_ID_Stage.sv ID_EX_Stage.sv EX_MEM_Stage.sv MEM_WB_Stage.sv fwd_logic.sv hazard_detection.sv top.sv
	./tb/	program_counter_tb.sv inst_mem_tb.sv data_mem_tb.sv reg_file_tb.sv imm_gen_tb.sv alu_logic_tb.sv branch_comp_tb.sv control_unit_tb.sv IF_ID_tb.sv ID_EX_tb.sv EX_MEM_tb.sv MEM_WB_tb.sv fwd_logic_tb.sv hazard_detection_tb.sv top_tb.sv



RISC-V Data and Control Paths

A rough estimate of pipeline datapath with hazard detection and forwarding is shown in the figure below.



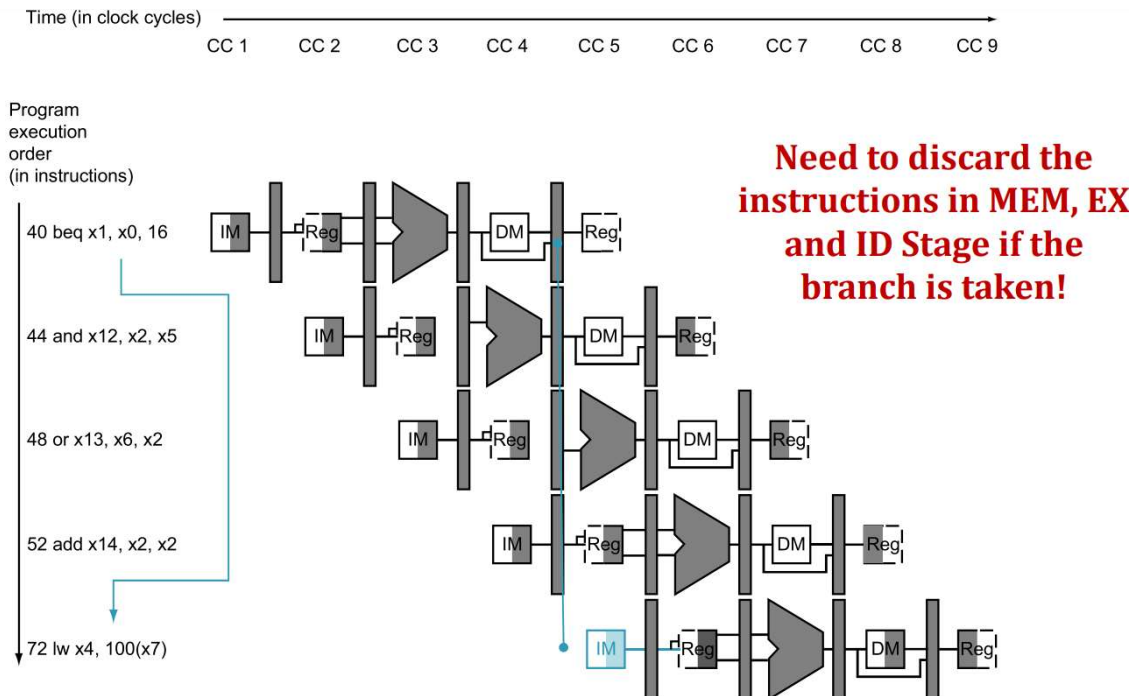
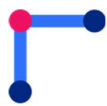
Pipelining: Control Hazards

A control hazard may occur when the flow of execution of a processor becomes unexpected. In the pipeline designed previously, a control hazard may occur in a branch instruction as shown in the figure below.

Branches cause control hazards because branch decision is delayed, and the processor continues execution until the decision has been made. In other words, a branch predictor exists with the assumption that branch is not taken.

In the case that branch needs to be taken, the processor must flush all the instructions following the branch instruction. To flush, you must disable register and memory writes, however, you may not disable writing of pipeline registers. This can be done in a similar manner to disabling RegWrite in Data Hazards.

Specifically, the data needs to be flushed in IF/ID and ID/EX Pipeline Registers.



Task

Implement additional logic to cater control hazards, so that branches could be taken correctly, without unexpected changes in state elements.

References

1. Computer Architecture and Organization (RISC-V Edition), David Patterson, John Hennessy
2. University of California, Berkeley (CS61C)